1 1 4 1 4 1

Token-Efficiency Research — § . . (UNCONFIRMED slot, verified free)

Revision: 1 **Last modified:** 2026-06-09T08:30:00Z **Status:** research **Scope:** Universal, decoupled, reusable token-efficiency solution for any AI-coding-agent-driven project under this constitution. **Authority:** constitution submodule. Drafted under §11.4.6 (no-guessing — every external claim carries a source URL; every project-specific number is measured, not estimated) + §11.4.8 (deep-web-research-before-implementation).

> **Anchor-slot note.** §11.4.139 is the highest live anchor in `Constitution.md`. §11.4.140 is reserved by the in-progress `action_prefix_system` work (its `RULE_DRAFT.md` claims literal `11.4.140`). The next genuinely-free slot is **§11.4.141** — used by this work. Verified 2026-06-09 via `grep -oE "§11\.4\.1[0-9][0-9]" Constitution.md | sort -u | tail` (highest = 132–139) and `grep -rln "11\.4\.140" .` (only action_prefix_system).

. The goal, stated precisely

Cut an AI coding agent's token spend (input **and** output) to **30–40% of current** — a **60–70% reduction** — WITHOUT degrading quality, performance, or safety, and WITHOUT breaking any existing mechanism (the §11.4 covenant family, the propagation gates, the doc-sync engine, CodeGraph, the parallel-stream methodology).

This is a **cost** target, not a context-size target. The two are different: prompt caching does not shrink the prompt at all, yet it removes 90% of the *cost* of the unchanged prompt. Most of the target is reachable by changing what gets *charged*, not what gets *sent*.
[1]

. Baseline — this project's measured cost drivers (FACT, not estimate)

Measured 2026-06-09 on this working tree (`wc -c` / `wc -l` ; token estimate = bytes ÷ 4, the standard English-text heuristic — exact counts come from the §MEASUREMENT harness's `count_tokens` calls, never `tiktoken` , per the token-counting source below):

Always-loaded file (re-sent every turn)	Bytes	Lines	~Tokens (B÷4)
`CLAUDE.md` (consumer)	361,977	3,078	~90,500
`constitution/CLAUDE.md` (imported via `@constitution/CLAUDE.md`)	318,509	3,119	~79,600
Subtotal — static governance, EVERY turn	**680,486**	**6,197**	**~170,000**
`AGENTS.md` (consumer)	212,549	2,181	~53,100
`QWEN.md` (consumer)	84,467	310	~21,100
`constitution/Constitution.md` (referenced, not always inlined)	781,977	9,167	~195,000

The consumer `CLAUDE.md` Applied-Fixes table alone has **112 numbered rows** (`grep -cE '^\| *[0-9]+ *\|'`), each a multi-hundred-word paragraph. The `§11.4.X` anchor blocks are restated verbatim in full.

The dominant driver is mechanically confirmed by an external incident on the exact same pattern: Claude Code issue **#24147 — "Cache read tokens consume 99.93% of usage quota - architectural scaling issue with**

CLAUDE.md re-reads" (`https://github.com/anthropics/claude-code/issues/24147`). The DEV deep-trace "Where Do Your Claude Code Tokens Actually Go? We Traced Every Single One" (`https://dev.to/slima4/where-do-your-claude-code-tokens-actually-go-we-traced-every-single-one-423e`) reaches the same conclusion: the always-loaded instruction context, re-charged per turn, is where the tokens go.

Conclusion (FACT): ~170K tokens of essentially-static governance is reprocessed on every single turn. At Opus 4.8 input pricing ($5.00 / 1M tokens — `shared/models.md` via the `claude-api` skill), 170K tokens = **$0.85 of input per turn just for the unchanged instructions**, before the agent reads a single file or emits a single token. Over a multi-hundred-turn development cycle this is the single largest line item. It is also the most cacheable content in the entire system: it changes only when governance is amended (rarely, and always at a commit boundary).
[2]

. The levers — each cited, with an estimated %-reduction and per-agent applicability

Lever A — Prompt caching of the static governance (THE BIGGEST LEVER)

Mechanism (from the authoritative `claude-api` skill `shared/prompt-caching.md` + Anthropic docs):

- Prompt caching is a **prefix match**. `tools` → `system` → `messages` render order. A `cache_control: {type: "ephemeral"}` breakpoint on the last block of the stable prefix caches everything before it.
- **Cost model (load-bearing):** cache **reads cost ~0.1× base input price** (a 90% discount on the cached portion); cache **writes cost 1.25× for the 5-minute TTL, 2× for the 1-hour TTL**. Break-even is **two requests for 5-min TTL** (1.25× write + 0.1× read = 1.35× vs 2× uncached) and three for 1-hour.
- Verify via `usage.cache_read_input_tokens` / `usage.cache_creation_input_tokens`. If reads are zero across identical-prefix requests, a silent invalidator is present.

- Minimum cacheable prefix on Opus 4.8 / Sonnet / Haiku = 4096 tokens (`shared/prompt-caching.md` table). Our 170K-token governance is ~40× over the floor.

Sources: Anthropic "Prompt caching with Claude" (`https://www.anthropic.com/news/prompt-caching`) — *"reducing costs by up to 90% and latency by up to 85% for long prompts"*; Claude API prompt-caching docs (`https://platform.claude.com/docs/en/build-with-claude/prompt-caching`); `prompt-caching.ai` — *"90% Token Savings for Claude Code"* (`https://prompt-caching.ai/`); Finout 2026 Anthropic pricing guide (`https://www.finout.io/blog/anthropic-api-pricing`).

Estimated %-reduction (whole-cycle, INPUT side): The ~170K static governance is ~60–80% of per-turn input tokens in a typical turn that reads one or two files. Caching removes 90% of that portion's cost. Net input-cost reduction from this lever alone: **~50–65%** of total input cost (the dominant component), provided the cache stays warm (continuous traffic re-warms it; gaps > TTL need a `max_tokens: 0` pre-warm or the 1-hour TTL).

Per-agent applicability: | Agent | Supports the lever? | How | |---|---|---| | Claude Code | YES (native, on by default for the system/instruction prefix) | The harness already caches the system prompt + CLAUDE.md prefix. The project's job is to **not invalidate it** (no per-turn volatile bytes ahead of the breakpoint) and to keep governance byte-stable within a session. | | Cursor / Aider / any Anthropic-SDK caller | YES | `cache_control` on the last governance block; reuse the exact prefix on every call. | | Gemini CLI | YES (different API) | Gemini *context caching* — auto when using API-key auth, reuses prior system instructions + context (Gemini CLI "Token Caching and Cost Optimization", `https://google-gemini.github.io/gemini-cli/docs/cli/token-caching.html`). | | Qwen Code | YES | Gemini-CLI fork → inherits the same context-cache machinery; the hybrid Qwen+Gemini study reports 36–83% token reduction at 59–90% cache efficiency (`https://dev.to/samestrin/maximizing-ai-efficiency-building-hybrid-workflows-with-qwen-code-and-gemini-cli-l4c`). | | OpenAI Codex / OpenRouter-routed | YES | OpenAI/OpenRouter prompt caching (`https://openrouter.ai/docs/guides/best-practices/prompt-caching`). |

Risk to existing mechanisms: NONE. Caching is **transparent** — the model sees byte-identical input whether served from cache or recomputed. No rule text is removed, summarized, or weakened; every §11.4.X anchor is still present verbatim

in the prefix. This is the safest possible lever: it changes only billing, never behaviour. The only failure mode is a *silent invalidator* (a timestamp / UUID / unsorted-JSON ahead of the breakpoint) which costs money but never corrupts behaviour — and is detectable via `cache_read_input_tokens == 0`.

Lever B — Restructure governance into a thin always-loaded INDEX + on-demand detail (progressive disclosure)

Mechanism (context-engineering / progressive-disclosure literature): Load a concise **index** at session start (one line per rule/fix: anchor id + 1-sentence summary + path to the full text). Fetch the full anchor body / fix paragraph only when a task actually touches it. This is the "index-first" / "metadata-then-body" pattern that `agents.md` + Claude Code Skills already use (a Skill's short description sits in context; the full `SKILL.md` is read on demand — `shared/agent-design.md` "Skills").

Sources: "Progressive Disclosure ... control context (and tokens) in AI agents" (`https://medium.com/@martia_es/progressive-disclosure-the-technique-that-helps-control-context-and-tokens-in-ai-agents-8d6108b09289`); MindStudio "Progressive Disclosure in AI Agents" (`https://www.mindstudio.ai/blog/progressive-disclosure-ai-agents-context-management`); TokenOptimize "Context Engineering" — repo-level instruction files saw **~17% lower output tokens and ~29% lower median runtime** on PR-sized tasks (`https://www.tokenoptimize.dev/guides/context-engineering-reduce-token-usage`); Elastic "Knowledge Indicators" — pre-computed context **cut agent token cost by up to 75%** and lifted accuracy 60%→92% (`https://www.elastic.co/search-labs/blog/pre-computed-context-llm-agent-costs`).

Estimated %-reduction: The 112-row Applied-Fixes table + full anchor bodies are ~120K of the 170K. An index restates each in ~1 line: ~112 fix-rows × ~80 chars + ~90 anchors × ~120 chars ≈ ~20K bytes ≈ ~5K tokens. **The always-loaded payload drops from ~170K to ~25–35K tokens — an ~80% cut in the static-context size.**

Interaction with Lever A (critical): Lever B and Lever A are **complementary, not redundant, but they trade off**. Caching makes the 170K *cheap to re-send* (0.1×); indexing makes it *small to send* (full price on 30K). On a warm cache, Lever A alone

already gets the 170K to ~17K-equivalent input cost — so Lever B's marginal cost saving on a warm cache is small. **Lever B's real value is (1) the cold-start / cache-miss case, (2) reducing the latency of the first turn, (3) freeing context-window headroom for actual work, and (4) agents/turns where caching is unavailable or cold.** Recommended composition: **cache the thin index (Lever A on the small payload) AND fetch detail on demand (Lever B)** — best of both.

Per-agent applicability: UNIVERSAL — it is a file-layout + retrieval discipline, not an API feature. Works identically on every agent because it is just "load `INDEX.md`, fetch `detail/<id>.md` when relevant."

Risk to existing mechanisms: LOW, but real and must be designed around. The full rule text MUST remain present *by reference* and reachable in one hop, and the propagation gates (`CM-COVENANT-114-N-PROPAGATION`) check for **literal anchor strings** in the consumer files. **Safety requirement:** the index entries MUST each carry the literal `11.4.N` anchor token so the propagation gates still pass, and the full bodies MUST stay in a tracked, gate-scanned location (e.g. `constitution/Constitution.md` remains the canonical full text; the consumer `CLAUDE.md` becomes the index that points at it). Done this way, no rule is lost and no gate breaks — the rules move from "inlined verbatim in the consumer" to "summarized in the consumer + canonical in the submodule," which is exactly what §11.4.35 already prescribes (consumer = thin extensions, submodule = canonical).

Lever C — Retrieval / CodeGraph-first over full-file loading

Mechanism: Answer structural questions ("where is X / what calls Y / what would break") with CodeGraph (§11.4.78/§11.4.79 — already mandated, already installed) instead of reading whole files; use grep-scoped reads instead of whole-file loads when only a region is needed. The harness already tracks file state, so re-reads are avoidable (this prompt's own tool guidance says so).

Sources: §11.4.78 / §11.4.80 (CodeGraph is already a constitution mandate — this lever *uses* the existing infrastructure, no new dependency); TokenOptimize "LLM Token Optimization Strategies" (`https://www.tokenoptimize.dev/guides/llm-token-optimization-strategies`); the lumen MCP `semantic_search` is also wired in this environment (PreToolUse hook actively recommends it over grep/glob/find).

Estimated %-reduction: Retrieval-over-full-file is workload-dependent. On structural/navigation turns it replaces a 2–10K-token file read with a sub-1K structural answer. Across a cycle, **~10–25% of the *read* (non-governance) input** depending on how navigation-heavy the work is. It does NOT touch the governance driver — it is additive to A/B.

Per-agent applicability: Claude Code (CodeGraph MCP + lumen MCP), and any agent with an MCP/semantic-search surface. Gemini/Qwen have grep + 1M context (less need, same technique available). Genericised: "prefer structural index queries to whole-file loads."

Risk to existing mechanisms: NONE. CodeGraph is read-only and already trusted by the constitution (§11.4.78 "trust codegraph results"). No behaviour change.

Lever D — Subagent model-tiering (cheaper model for search/grep/status; reserve the strong model for reasoning) + output-to-file

Mechanism: Route mechanical subagent work (search, grep, status, simple edits, log-scraping, doc-export) to a Haiku-class model; reserve Opus for reasoning/architecture/review. Subagents write large output **to a file** and return a short pointer, instead of dumping a 350–520K-token transcript into the conductor's context. The constitution already runs subagent-driven-by-default (§11.4.20/§11.4.70) and parallel streams (§11.4.58/§11.4.103) — this lever adds the *tier* + *output-discipline* to those existing flows.

Sources: MindStudio "Manage Token Costs ... Haiku Sub-Agents and Scope Bounding" (`https://www.mindstudio.ai/blog/manage-token-costs-claude-code-dynamic-workflows`) — *"Teams report 40–85% token reduction using tiering tactics, with the biggest single subagent cost lever being using a lighter model for all subagents"*; Claude Code sub-agents docs (`https://code.claude.com/docs/en/sub-agents`) — the `model` field accepts `sonnet` / `opus` / `haiku` / `inherit` ; Augment "AI model routing guide" (`https://www.augmentcode.com/guides/ai-model-routing-guide`); `shared/agent-design.md` "Caching for Agents" — *"Spawn a subagent with the cheaper model for the sub-task; keep the main loop on one model"* (also preserves the main loop's cache). Pricing deltas (`shared/models.md` via `claude-api`): Opus 4.8 $5/$25 per 1M, Sonnet 4.6 $3/$15, **Haiku**

4.5 $1/$5 — Haiku input is **5× cheaper than Opus, output 5× cheaper**; the routing literature cites up to 12× input / 9.6× output savings vs Sonnet 4 for Haiku-appropriate tasks.

Estimated %-reduction: If ~half the subagent volume is Haiku-appropriate (search/grep/status/export), tiering those to Haiku saves ~80% of *their* cost. Subagent work is a large share of total spend under the parallel-stream methodology (3+ background streams continuously). Whole-cycle reduction from tiering + output-to-file: **~20–40%** of total cost, concentrated on the subagent fleet. Output-to-file alone removes the 350–520K-token transcript re-ingestion — that is a pure, large win on the conductor's input.

Per-agent applicability: Claude Code (subagent `model:` field + Task tool). Gemini/Qwen hybrid chaining does the same cross-model (Gemini = cheap context discovery, Qwen = synthesis) — 36–83% reduction cited above. Cursor/Aider support model selection per-request. Genericised: "tier the model to the task; persist large subagent output to disk."

Risk to existing mechanisms: LOW — must respect two existing rules. (1) §11.4.50 deterministic consistency and the anti-bluff covenant: a cheaper model on a *reasoning* or *verdict* path could lower quality — so tiering is restricted to **mechanical, non-judgment** subagent work (search/grep/status/export), never to a step that produces a PASS/FAIL verdict, a fix design, or a code review (§11.4.125 reviewer stays strong-model). (2) §12.6 60% memory ceiling + the parallel-stream caps are unchanged. With those guards, tiering cannot degrade quality — the strong model still owns every decision; only the legwork is cheap.

Lever E — Output-token reduction (terse, structured responses; low effort for mechanical subagents)

Mechanism: Terse conductor status (one line per milestone, no restatement of unchanged context); structured/JSON responses where a sink consumes them; `output_config: {effort: "low"}` on mechanical subagents (the `claude-api` skill notes low effort → *"fewer and more-consolidated tool calls, less preamble, terser confirmations"*). Avoid re-stating the rules back to the operator.

Sources: `claude-api` skill §Effort + `shared/agent-design.md`; Augment routing guide (output discipline); Finout pricing (output is 5× input price on Opus — every output token saved is worth 5 input tokens).

Estimated %-reduction: Output is typically 10–30% of token *count* but, at 5× the price, a larger share of *cost*. Terse output + low-effort mechanical subagents cut output tokens ~30–50% on those paths → **~10–20% of total cost**.

Per-agent applicability: UNIVERSAL (it is a prompting/response discipline + an effort flag). Opus 4.8/4.7 honour `effort`; other agents have equivalents or respond to a terse-output instruction.

Risk to existing mechanisms: NONE for the terse-status part; LOW for effort. Terseness is a presentation change, not a behaviour change — the captured-evidence artefacts (§11.4.5/§11.4.69) are unchanged; only the conductor's prose narration shrinks. `effort: "low"` is restricted to mechanical subagents (same guard as Lever D) so no reasoning path is degraded.

Lever F — Tool-call efficiency (batch independent calls; never re-read)

Mechanism: Issue independent tool calls in one block (the harness already supports parallel tool calls); never re-read a file the harness already tracks; reuse persisted tool outputs. This prompt's own operating instructions mandate batching independent calls.

Sources: `shared/tool-use-concepts.md` (parallel tool use is default; `disable_parallel_tool_use` is opt-out); this environment's tool guidance.

Estimated %-reduction: Small but free — **~3–8%** by eliminating redundant reads and round-trips. Additive to all others.

Per-agent applicability: UNIVERSAL.

Lever G — Compaction / context-editing for long-running sessions (defensive, not primary)

Mechanism: Server-side compaction (`compact-2026-01-12` , Opus 4.8/4.7/4.6/ Sonnet 4.6) summarizes earlier context when nearing the window; context-editing prunes stale tool results/thinking. Keeps a long session from re-paying for an ever-growing transcript.

Sources: `claude-api` skill §Compaction + `shared/agent-design.md` .

Estimated %-reduction: Situational — prevents *growth* of cost on very long sessions rather than cutting the baseline. Counts as a guard that keeps the other levers' gains from eroding over a 100+ turn cycle. **~5–15%** on long sessions, ~0 on short ones.

Per-agent applicability: Claude Code / Anthropic SDK; Gemini/Qwen have their own compaction. Genericised: "compact/prune long transcripts."
[3]

. Open-source tooling for token optimization (cited)

- **Token measurement (the harness's backbone):** Anthropic `count_tokens` endpoint (`https://platform.claude.com/docs/en/build-with-claude/token-counting`) — the *only* accurate Claude tokenizer; the `claude-api` skill `shared/token-counting.md` explicitly forbids `tiktoken` (undercounts Claude by 15–20%+). The `usage` object (`cache_read_input_tokens` / `cache_creation_input_tokens` / `input_tokens` / `output_tokens`) is the per-request ground truth.
- **Usage tracking corroboration:** `she-llac/claude-counter` browser extension (`https://github.com/she-llac/claude-counter`); AgentsRoom per-session token tracker with cache-hit-rate (`https://agentsroom.dev/features/claude-code-token-usage`); Shipyard "Claude Code tokens" (`https://shipyard.build/blog/claude-code-tokens/`).
- **The problem, externally documented:** Claude Code issue #24147 (CLAUDE.md re-reads consume 99.93% of cache-read quota) — the canonical statement of exactly this project's driver.

- **Retrieval infra already in tree:** CodeGraph (`@colbymchenry/codegraph` , mandated §11.4.78) + lumen MCP `semantic_search` (active in this environment).

. Ranked levers with estimated %-reductions

Rank	Lever	Cost component hit	Est. cycle-cost reduction (standalone)	Risk	Per-agent reach
1	**A — Prompt-cache the static governance**	Input (the 170K driver)	**50–65%**	NONE (transparent)	Claude Code (native) + all SDK/Gemini/Qwen
2	**D — Subagent model-tiering + output-to-file**	Subagent input+output	20–40%	LOW (mechanical-only)	Claude Code + Gemini/Qwen hybrid
3	**B — Thin INDEX + on-demand detail**	Static-context *size* (cold-start, latency, window)	80% size cut; ~5–15% *cost* on warm cache, more on cold	LOW (keep anchors literal + canonical text by reference)	UNIVERSAL
4	**C — Code-Graph/retrieval over full-file**	Read input	10–25%	NONE	Claude Code + MCP agents
5	**E — Output-token reduction (terse + low-effort)**	Output (5× price)	10–20%	NONE/LOW	UNIVERSAL
6	**F — Tool-call batching / no re-reads**	Round-trip input	3–8%	NONE	UNIVERSAL

Rank	Lever	Cost component hit	Est. cycle-cost reduction (standalone)	Risk	Per-agent reach
7	**G — Compaction/ context-editing (long sessions)**	Transcript growth	5–15% (long) / 0 (short)	NONE	Claude Code + Gemini/ Qwen

The single biggest lever is A (prompt caching the static governance). It alone gets the largest share of the target because the governance is both the dominant cost and the most cacheable content in the system. It is also the safest (transparent, behaviour-identical, every rule still present verbatim).

See DESIGN.md for the composed solution and the realistic combined number; MEASUREMENT.md for the proof methodology; RULE_DRAFT.md for the §11.4.141 anchor.
[5]

1 1 4 6

. Honest combined-reduction estimate (per § . . — no over-claiming)

The levers are **not purely additive** (A and B overlap on the same governance bytes; D and E overlap on subagent output). Composed conservatively (warm-cache steady state, typical mixed dev cycle):

- A alone: input cost → ~35–50% of baseline (cached governance at 0.1×).
- A + D + E + C + F + G: total cost → **~30–40% of baseline (a 60–70% reduction) is ACHIEVABLE AND REALISTIC**, contingent on three measurable conditions:
    1. **The cache stays warm** (continuous traffic, or `max_tokens: 0` pre-warm, or 1-hour TTL across gaps) — verified by `cache_read_input_tokens > 0`.

2. **No silent invalidator** sits ahead of the governance breakpoint — verified by the same field.
3. **Mechanical subagent volume is materially Haiku-routed** — verified by per-subagent model accounting.

If the cache is frequently cold (bursty, gap-heavy sessions) OR caching is unavailable on a given agent, the **safe floor without caching** is A-absent: B + C + D + E + F ≈ **45–55% of baseline (a 45–55% reduction)** — still large, driven by the index size-cut + tiering + output discipline. **This is the honest worst-case number to quote when caching cannot be relied on.** The 60–70% target is the warm-cache case and is the design default, but the rule (RULE_DRAFT.md) is written to *require the measured proof*, not to assert the number — so a project that cannot keep the cache warm still complies by hitting its *measured* best safe reduction, never by bluffing the headline figure.

Sources

- Prompt caching with Claude (Anthropic)
- Prompt caching — Claude API Docs
- prompt-caching.ai — 90% Token Savings for Claude Code
- Anthropic API Pricing 2026 — Finout
- Pricing — Claude API Docs
- Token counting — Claude API Docs
- Claude Code issue #24147 — cache-read quota from CLAUDE.md re-reads
- Where Do Your Claude Code Tokens Actually Go? (DEV)
- Context Engineering: Reduce Token Usage — TokenOptimize
- LLM Token Optimization Strategies — TokenOptimize
- Progressive Disclosure in AI Agents (Medium)
- Progressive Disclosure in AI Agents (MindStudio)
- Knowledge Indicators: cutting LLM agent costs — Elastic
- Manage Token Costs — Haiku Sub-Agents and Scope Bounding (MindStudio)
- Claude Code sub-agents docs

- AI Model Routing Guide — Augment
- Gemini CLI Token Caching and Cost Optimization
- Hybrid Qwen Code + Gemini CLI workflows (DEV)
- Prompt Caching best practices — OpenRouter
- `claude-api` skill (bundled): `shared/prompt-caching.md`, `shared/token-counting.md`, `shared/agent-design.md`, `shared/models.md`, `shared/tool-use-concepts.md` — Opus 4.8 $5/$25, Sonnet 4.6 $3/$15, Haiku 4.5 $1/$5 per 1M; cache read 0.1×, write 1.25× (5m) / 2× (1h); 4096-token min prefix on Opus.